Pi Motion Cam: Standard Operating Procedure (SOP) & Best Practices

Applies to: the Raspberry Pi 5 camera service (`main.py` + `modules/`), the Pi dashboard (`webui/`, port 8080), and the browser web app v8 (https://vibhvesh.github.io/pi-motion-cam/). Everything here is based on the code and `config.yaml` in this repo. Setting names are written exactly as they appear in `config.yaml`.

---

1. Purpose & scope

- **Purpose:** run a local smart camera that detects motion and people, logs events with snapshots, takes face-recognition attendance, counts people in and out, watches zones and lines, reads number plates and sends Telegram alerts. All of this runs on-device, with no cloud video.
- **In scope:** installing, configuring, enrolling people, daily operation, tuning, incident handling, privacy, security and troubleshooting.
- **Out of scope:** life-safety use. `fire_smoke` and `fight` are **EXPERIMENTAL heuristics**: `fire_smoke` is not a smoke alarm, and `fight` is not a reliable violence detector. Never use either as the only safety system.

2. Roles

Role	Who	Responsibilities
Admin	1–2 trusted people	Installs and upgrades the Pi, owns the `config.yaml` and dashboard password, sets up Telegram, draws zones and lines, enables modules, handles backups, data retention, privacy requests and security
Operator	Reception, security or facility staff	Daily checklist, labels the unknown-faces queue, reviews events and alerts, exports CSVs, cleans the lens, escalates problems to the Admin

The dashboard has a **single shared password** (`web.password`). There are no per-user accounts, so share it only with Admins and Operators, and change it when someone leaves.

3. Hardware & placement best practices

Pi 5 - Use the official 27 W USB-C PSU. Undervoltage causes random crashes and camera errors. - Use active cooling (the official Active Cooler or a fan case). Detection, OCR and face recognition keep the CPU busy, and the Pi throttles at about 80–85 °C. Aim to stay **below 70 °C** (watch it in Dashboard → 🖥️ System). - Prefer a good A2 microSD or an NVMe/SSD. Events, snapshots and CSVs are written all the time. - Use wired Ethernet if possible. Keep the Pi and camera on a UPS if the site has power cuts. - Camera: connect the ribbon with the power **off**, then check it with `rpicam-hello -t 2000`.

General camera rules - **Avoid backlight.** Never point the camera at a window, glass door or the sun. Faces turn into silhouettes, and motion and tamper alerts go off. - Keep lighting steady. Flickering lights, TVs, moving curtains, trees and car headlights are the main causes of false motion. - At night the Pi brightens frames and lowers the detection threshold (`detector.night`: `dark_brightness`, `enhance`, `confidence_scale`). This only helps up to a point. Add an IR illuminator with a NoIR camera, or a soft white light. Note that face recognition and colour-based fire detection work poorly in IR or greyscale video. - Mount the camera solidly. Vibration or knocks set off `tamper` (`scene_change`).

Module	Placement
`attendance`	At the **entrance, at face height (~1.5–1.7 m)**, looking straight at people walking in. Pick a spot where people pause or walk towards the camera, 1–3 m away, with even light on faces. Avoid steep downward angles.
`counter` (in/out)	**Overhead or high above the doorway**, looking down and across the passage, so people cross the `line` clearly one at a time. Keep the whole doorway in frame.
`zone_intrusion`, `loitering`, `occupancy`	High corner (2.5–3 m) with the whole area in view. People's feet must be visible inside the zone.
`perimeter`	Looking along or across the fence or boundary, with the tripwire `line` drawn where people or animals cross. Animals need `detector.type: mobilenet_ssd`.
`plates`	**Close, shallow angle (≤ 30° off-axis), daylight or good light**, at a gate or barrier where vehicles slow down. Plates should be roughly ≥ 100 px wide in the 2304×1296 frame. Avoid headlight glare at night.
`fall`	A side or angled view of the room floor, not straight down (it uses box shape and pose).
`fire_smoke`	Wide view of the risk area, with no flame-coloured décor, sunsets or flashing lights in frame.

4. Installation & upgrade

1. Use Raspberry Pi OS Bookworm (64-bit), connect the camera, and run `rpicam-hello -t 2000`.
2. Install with only the options you need: `bash curl -sSL https://raw.githubusercontent.com/vibhvesh/pi-motion-cam/main/get.sh | bash -s -- --dnn --ocr --face` | Option | Adds | |---|---| | `--dnn` | MobileNet-SSD detector for people, animals and vehicles. Set `detector.type: mobilenet_ssd`. | | `--face` | `face_recognition`/dlib for attendance. The build takes 30+ minutes. | | `--ocr` | Tesseract OCR for number plates | | `--pose` | MoveNet pose model for better fall and fight detection. Set `pose.enabled: true`. | | `--no-service` | Skips installing the systemd services |
3. The installer creates the `venv`, installs and starts `pi-motion-cam` (camera) and `pi-motion-cam-web` (dashboard), and prints **`Dashboard: http://<pi-ip>:8080`**.
4. **Upgrade:** run the same one-liner again. It does a `git pull`, keeps your edited `config.yaml`, saves the new defaults as `config.yaml.upstream`, and re-runs the installer. Compare the two files for new settings.
5. **Before every upgrade**, back up `config.yaml` and `known_faces/` (see §8).

5. First-time configuration (Admin)

Open `http://<pi-ip>:8080` from a phone or PC on the same network. 1. **Set the dashboard password:** ⚙️ Settings → Password → Save. This stores it as `web.password`; the `PMC_WEB_PASSWORD` environment variable can override it. 2. **Set up Telegram:** - In Telegram, create a bot with @BotFather and copy its token. - Send the bot any message. - Get your chat id from `https://api.telegram.org/bot<TOKEN>/getUpdates`. - In ⚙️ Settings, enter the token and chat id, tick Enabled, Save, then press **Send test**. 3. **Detector:** to detect animals, vehicles or plates, set ⚙️ Settings → detector `type: mobilenet_ssd` (needs `--dnn`) and click **Save & restart**. Otherwise `hog` detects people only. 4. **Draw zones and lines** (✏️ Zones): - Press 📸 New snapshot and pick the module. - For a line (`counter`, `perimeter`, `plates`), tap 2 points. For a zone (`zone_intrusion`, `loitering`, `occupancy`, `perimeter`), tap 3 or more points and give it a name. Then Save. - Coordinates are stored normalised (0–1). - Counter: if IN and OUT come out reversed, flip `in_side` (`negative`/`positive`). 5. **Enable modules** (🧩 Modules): switch on only what you need, one at a time, and check the 📜 Events tab for about a day before enabling the next. - Changes hot-reload within about 2 s. - Camera size or fps changes need **Save & restart**. 6. **Set schedules:** every module accepts `active_hours` (`""` means always; for example `"20:00-06:00"`, which can wrap past midnight), plus `cooldown_seconds` and `telegram: true/false`. 7. **Daily summary:** `summary.enabled` and `summary.send_time` (default `21:00`) control the daily Telegram summary.

5a. Testing settings with a file (before going live, and after every tuning change)

Use recorded clips from the **actual camera position** so a setting change can be checked without waiting for a real event. 1. **Pi dashboard:** go to 🧪 Test file, pick a clip (≤ 200 MB, jpg/png/mp4/avi/mov/mkv/webm) and press ▶ Run test. You can tick Send Telegram alerts, which is off by default. - The test runs in a separate low-priority process, and the live camera keeps recording. - It uses the modules and settings currently in `config.yaml`, and the real `known_faces/`. - Results are written to a temp folder (`/tmp/pi-motion-cam-tests/`, last 5 runs kept). They never go into the real `events/`, `attendance/` or `plates/` logs. 2. **Web app:** press 📁 Test with file, or drop a file on the video. - For still images, set Repeat frames (default 10) so tracking and hold timers can trigger. - Events are tagged `source=file:<name>`. Clear them afterwards in Events → Clear if you don't want them mixed with live events. 3. Check that the expected events fire (and no false ones). Adjust settings and re-run the same clip until you're happy, then watch the live result for a day. 4. **Keep a small library of test clips:** a person entering, a crowd, a vehicle at the gate, an empty scene at night. Re-test after every upgrade. 5. Time-based settings use the **file's own time** on the Pi (frame number ÷ fps). `active_hours` is ignored during tests. 6. Test clips contain personal data. Delete them when you're done, and never use footage of people who haven't been informed (§11). 7. There is no helmet/vest (PPE) module, so PPE can't be tested.

6. Enrollment SOP (attendance)

Photo quality - Take **5–10 photos per person**: frontal, about ±20° left and right, slightly up and down, with and without glasses, in the entrance's real lighting. - One clear face per photo, in focus, face at least about 150 px wide. No sunglasses, masks or heavy backlight. - Photos from the actual entrance camera match best. Face crops labelled from the unknown queue are ideal extra samples.

Naming conventions - Use one consistent name per person, for example `Rahul Sharma` or `rahul.sharma`, ideally with an employee id if names clash (`Rahul Sharma 1042`). - The dashboard stores photos as `known_faces/<name>/<name>_N.jpg`. The older flat layout (`alice.jpg`, `alice_2.jpg`, all counted as alice) still works. - Dashboard names are cleaned to letters, digits, spaces and `- _ .`, up to 60 characters.

Enroll 1. Get **consent** first (see §11). 2. Go to Dashboard → 🧑‍💼 Attendance → Enroll photos, type the name, pick the photos and click Upload. 3. With `face_recognition` installed, photos with no detectable face are rejected. Check the ✅/❌ list. 4. The camera service re-reads `known_faces/` automatically; no restart is needed.

Labeling the unknown queue (Operator, daily) - New unrecognised faces are saved to `unknown_faces/` and trigger a Telegram message: "Unknown person — please label in admin panel". - In Dashboard → Attendance → Unknown faces queue, type the name and press **Enter** or **Save**. The photo moves into `known_faces/<name>/`. - Use **Discard** for visitors, blurry crops or non-faces. Never label a face you are not sure about, because a wrong label teaches the system the wrong person. - In the browser web app (v7), the queue works the same way but is stored in that browser only.

Removing people (leavers, or consent withdrawn) - Go to Dashboard → Attendance → Known people → **Delete**. This deletes every photo of that person. - Old rows in the `attendance/*.csv` files are kept. Delete or redact them by hand if required (§11).

Tuning: `tolerance: 0.6` is the default. - If people get mixed up, lower it to 0.5–0.55. - If known people are often logged as Unknown, add more photos first, then raise it slightly. - `relog_gap_minutes` (default 30) controls when a new IN row starts.

7. Operating checklists

Daily (Operator) - [ ] Dashboard header shows `pi-motion-cam: active`, and 🎥 Live shows a current picture. - [ ] 🖥️ System: CPU temperature below about 70 °C, disk free above 15 %. - [ ] Label or

discard the **unknown faces queue**. - [ ] Review 📜 Events (filter severity `alert` / `warning`) and close out any incidents (§10). - [ ] Check that the Telegram daily summary arrived.

Weekly (Operator → Admin) - [ ] Export CSVs from Attendance, Events and Plates for the week, and archive them as policy requires. - [ ] Clean the lens with a microfibre cloth. Check the mount, the angle (compare with a snapshot) and the IR or lights. - [ ] Review false alarms and adjust thresholds, zones or `active_hours` (§9). - [ ] Press **Send test** in Telegram settings.

Monthly (Admin) - [ ] **Back up** `config.yaml` and `known_faces/` (plus `attendance/`, `plates/`, `events/` if they must be kept): `bash cd ~/pi-motion-cam && tar czf ~/pmc-backup-$(date +%F).tgz config.yaml known_faces attendance plates events` Copy the archive off the Pi and keep it encrypted, because it contains biometric data. - [ ] Apply **retention** (§11): no automatic deletion exists in the code, so delete old `captures/`, `events/`, `unknown_faces/` and so on by hand or with cron, for example: `find ~/pi-motion-cam/captures -name '*.jpg' -mtime +30 -delete` - [ ] Update the OS and the app: `sudo apt update && sudo apt full-upgrade`, then re-run the `get.sh` one-liner. - [ ] Review who knows the dashboard password, and rotate it if needed. - [ ] Re-enroll people whose appearance has changed a lot.

8. Data locations (on the Pi, under `~/pi-motion-cam/`)

Path	Contents
`config.yaml` (+ `config.yaml.bak` from the last dashboard save)	All settings, including the Telegram token and dashboard password: **sensitive**
`known_faces/`	Enrollment photos: **biometric, sensitive**
`unknown_faces/`	Face crops of unrecognised people
`attendance/YYYY-MM-DD.csv`	name, first_seen_in, last_seen_out
`events/YYYY-MM-DD.csv` + `captures/`	Every event, with annotated snapshots and crops
`plates/YYYY-MM-DD.csv`	plate, direction, time, confidence
`summaries/YYYY-MM-DD.json`	Daily summary
`/dev/shm/pi-motion-cam/`	Live frame and status for the dashboard (RAM only)

The browser web app stores its settings, faces, events and attendance in that browser's localStorage. Use its own Export buttons for those.

9. Per-module tuning & known false-alarm sources

Module	Main settings	Common false alarms → fix
`motion_alert`	`require_person`, `motion.min_area`, `motion.pixel_threshold`	Lighting changes, trees, insects at night → keep `require_person: true`, raise `min_area`
`attendance`	`tolerance`, `min_interval_seconds`, `relog_gap_minutes`	Wrong names → lower `tolerance`, add photos. Repeated unknowns of the same person from bad angles → improve placement
`counter`	`line`, `in_side`, `classes`	Double counts when people stop on the line or walk in groups → move the line to where people walk steadily; overhead view
`zone_intrusion`	`zones`, `classes`, `active_hours`	Zone edges touching walkways → shrink the zone; use `active_hours`
`perimeter`	`line`, `direction`, `classes`	Birds, cats, shadows → remove `bird`/`cat` from `classes`, set `direction`
`loitering`	`max_seconds`, `zones`	Staff working normally → zones only where loitering matters; raise `max_seconds`

Module	Main settings	Common false alarms → fix
`occupancy`	`max_people`, `hold_seconds`	Brief crowding → raise `hold_seconds`
`fall`	`down_seconds`, `drop_fraction`	People sitting on the floor, lying on sofas, bending → raise `down_seconds`; enable pose (`--pose`)
`fight` **(experimental)**	`hold_seconds`, `energy_threshold`, `proximity`	Hugs, sports, dancing, kids playing → raise `energy_threshold`/`hold_seconds`; always verify on video
`tamper`	`dark_threshold`, `flat_std`, `blur_threshold`, `scene_change`, `hold_seconds`	Lights switched off, sudden IR switch, camera knocked → raise `hold_seconds`; mount firmly
`fire_smoke` **(experimental)**	`min_fire_fraction`, `flicker_threshold`, `min_smoke_fraction`	Sunsets, orange clothes or lights, flashing displays, fog, steam, dust → raise thresholds; **not a smoke alarm**, keep real detectors
`animal` / `after_hours`	`classes`, `active_hours`	Pets, staff working late → adjust `classes`, `active_hours`
`plates`	`min_confidence`, `tries`, `indian_only`, `watchlist`, `out_after_minutes`	Misreads (0/O, 8/B) → better angle and light, `indian_only: true`, raise `min_confidence`. OUT times are estimates unless a `line` is set

General rules: - Raise `cooldown_seconds` for modules that send too many alerts. - Use `detector.confidence` (default 0.4) to trade missed detections against false ones. - Turn `telegram: false` on for log-only modules, such as `counter`, which is off by default.

10. Alert handling & incident response

1. **Receive:** a Telegram alert arrives with `[module] message` and a snapshot or crop.
2. **Verify** within an agreed response time (for example 5 minutes for `alert` severity): open 🎥 Live and 📜 Events (filter by module), and check the snapshot.
3. **Classify:**
4. **False alarm:** note the cause and tune it (§9) at the weekly review.
5. **Real, non-urgent** (unknown person, after-hours staff, watchlist plate seen): follow site policy; label or record it.
6. **Real, urgent** (intrusion, fall, fight, fire/smoke): act at once. Call security or emergency services first; the camera is not a substitute.
7. **Preserve evidence:** download the snapshot from `captures/` and export that day's `events` CSV (and `plates/attendance` if relevant). Copy them off the Pi before retention deletes them. Record who accessed them.
8. **Log** the incident (date, time, module, action taken, outcome) in your site register.
9. **Review:** if a module fails, either missing a real event or raising repeated false alarms, the Admin adjusts settings or placement and notes the change.

11. Privacy & legal

- **Notice & signage:** put clear signs at every monitored area (for example "CCTV with face recognition and number-plate reading in operation. Contact: …").
- **Consent:** get informed, written consent before enrolling anyone's face. Offer a non-biometric alternative for attendance where possible. Remove people promptly when they leave or withdraw consent (§6).
- **India: Digital Personal Data Protection Act, 2023 (DPDP Act)** and its Rules. Faces, images and plate-linked records are personal data. As the Data Fiduciary you should:
- have a lawful purpose and notice;
- collect only what you need;

- keep data accurate;
- store it securely;
- delete it when the purpose is served;
- honour access, correction and erasure requests;
- report breaches as the law requires. Take legal advice for your situation. Elsewhere, local laws (for example GDPR) apply.
- **Data minimisation:** enable only the modules you need, and avoid pointing cameras at private spaces (toilets, changing rooms) or into neighbours' property.
- **Retention:** the code keeps data until it is deleted. Set a written retention period, for example 30 days for snapshots and events, and as long as your policy needs for attendance, then apply it monthly (§7).
- **Access control:** only Admins and Operators get the dashboard password. Telegram alerts contain photos, so limit who is in the chat. Encrypt backups.

12. Security

- **LAN only.** The dashboard is plain HTTP. **Never port-forward port 8080** or expose it to the internet. For remote access, use a VPN such as WireGuard or Tailscale.
- **Always set `web.password`.** With it empty, anyone on the Wi-Fi can open the dashboard, change settings and see faces.
- Treat `config.yaml` and `config.yaml.bak` as secrets (Telegram token, password). Don't commit them to a public repo.
- The dashboard's sudo rights are limited to start/stop/restart of `pi-motion-cam` and reading its journal (`/etc/sudoers.d/pi-motion-cam`). Don't widen them.
- Change the default Pi user password, use SSH keys, keep the OS updated monthly, and put the Pi on a separate or IoT network if you can.
- If the Telegram token leaks, revoke it in @BotFather (`/revoke`) and enter the new one in ⚙️ Settings.

13. Troubleshooting

Symptom	Check / fix
Dashboard won't open	Is the Pi on the same network? `systemctl status pi-motion-cam-web`, then `sudo systemctl restart pi-motion-cam-web`. Check the IP with `hostname -I` and the port (`web.port`, default 8080).
🎥 Live shows "Camera service not running" / no stream	Header shows the service state. In 🖥️ System press Restart, or run `sudo systemctl restart pi-motion-cam`. Then check the logs (System → Show logs, or `journalctl -u pi-motion-cam -f`). Test the camera with `rpicam-hello`. Only one program can use the camera at a time.
Service keeps restarting	The logs usually show a YAML error in `config.yaml` (restore `config.yaml.bak`) or a missing optional package (for example "Module attendance needs an optional package": re-run the installer with `--face`).
Attendance module won't start on the Pi	`face_recognition` isn't installed: run `./install.sh --face`.
Web app: face models fail to load	Click **Retry loading face models**, check the connection, then hard-refresh (Ctrl+F5) or clear the site cache. Make sure the header shows **v8**.
Web app: can't type a name in the unknown queue (Edge)	Fixed in v7. Hard-refresh until the header says v8 (current).
Test file: "cannot decode"	Browser: use mp4 (H.264) or webm; mov only works where the browser supports its codec. Pi: OpenCV/FFmpeg must support the codec; convert it with `ffmpeg -i in.mov out.mp4`.

Symptom	Check / fix
Test file finishes with 0 events	No modules enabled, or a still image (no motion: use more Repeat frames and occupancy/zone modules). Check the process log in 🧪 Test file.
Test file: "A test is already running"	Only one Pi test runs at a time. Wait, or press ■ Stop.
Web app camera doesn't start	Must be HTTPS (the GitHub Pages URL); allow camera permission.
High CPU / temperature / slow	Lower `web.live_fps` (default 5) or `web.live_width`, raise `detector.idle_interval_seconds`, disable unused modules, turn off pose, and improve cooling. Close the 🎥 Live tab when not needed (the stream only runs while open).
Too many Telegram messages	Raise `cooldown_seconds` or set `telegram: false` per module; use `active_hours`.
No Telegram messages	⚙️ Settings → **Send test**, and read the error. Did you message the bot first? Is the chat id right? Is `events.telegram.enabled` set?
Plates not read	Need `detector.type: mobilenet_ssd` + `--ocr` + `modules.plates.enabled`; improve the angle and light; lower `min_confidence` carefully.
Disk full	Delete old `captures/` and `events/` (§7), or move to a larger SSD.
Settings not applied	Hot reload takes about 2 s; camera size/fps need **Save & restart**. The logs show `config.yaml reloaded`.

Useful commands:

```
sudo systemctl status|restart|stop pi-motion-cam pi-motion-cam-web
journalctl -u pi-motion-cam -n 200 --no-pager
vcgencmd measure_temp; df -h; vcgencmd get_throttled   # 0x0 = no undervoltage/throttling
./venv/bin/python main.py --summary                     # today's summary JSON
```